
Compass Architecture

Private eligibility firewall on oG. Two on-chain contracts, three off-chain services. Single-principal soulbound Agent INFTs. Bounded-disclosure receipts.

Layers

Frontend (app/) – Next.js 16 + Privy embedded wallet
(Privy wired in /onboard step 1; live behind
NEXT_PUBLIC_PRIVY_APP_ID, fixture timer in default build)

- Maria onboards, mints Agent, uploads VC, requests eligibility, views receipt, sees /clinic/subpoena



Off-chain Enclave (enclave/) – TypeScript node service

- SD-JWT VC issuer (mocked NGOs)
- SD-JWT VC verifier + policy evaluator
- 0G Storage upload (AES-256-GCM client-side encrypted)
- 0G Sealed Inference broker (TeeML / dstack TDX – receipt key bound into REPORTDATA, see honest-limits \$5b)
- Receipt construction per docs/schemas/receipt-v1.json



0G Storage

- Encrypted VC vaults
- Merkle-rooted permanent archive
- Off-chain content



0G Compute (Sealed Inference)

- TEE policy evaluation
- Receipt signed by key bound into TDX REPORTDATA (recovered signer match)



0G Chain (Aristotle mainnet – Galileo testnet for staging)

contracts/AgentRegistry.sol

- ERC-721 + soulbound _update
- mintAgent (zero-hash reject)
- updateMetadata (owner-gated)
- authorizeUsage (ERC-7857)
- verifyAttestation v1 stub

contracts/CompassHub.sol

- EIP-712 "Compass" v1
- IAgentRegistry binding
- consumeGrantAndIssueReceipt atomic; 8 sequenced validations + 2 effects + 2 events in 1 tx
- Policy registry (admin = first registrant)
- registerPolicy / deactivatePolicy
- transferOracleAdmin

Trust boundaries

Component	Trusted by	Trust assumption
Privy wallet	Maria	Embedded wallet doesn't leak key
AES-256-GCM	All	NIST-vetted; never nonce-reuses
SD-JWT VC issuer (NGO)	Verifier	Issuer keypair held honestly
oG Storage	All	Merkle-rooted archive returns same bytes
oG Sealed Inference TEE	Provider + verifier	dstack TDX measures the full VM image + Compose hash + container digests. Receipt-signing key derived inside the TEE; corresponding Ethereum address bound into the TDX quote's <code>report_data</code> field. Verifier runs Intel DCAP / dstack verifier and checks recovered signer matches embedded address — hardware-bound enclave-born key proof. See <code>docs/honest-limits.md</code> §5b.
AgentRegistry	CompassHub	<code>ownerOf(tokenId)</code> is canonical agent owner
CompassHub	Provider	Receipt log is append-only + dedup-checked

Single-principal model (target architecture; v2)

Maria's Privy-derived secp256k1 key is *intended* to play four roles. v1 wires role 1 (frontend onboarding step 1) only; roles 2–4 are the v2 wire-up.

1. EVM owner of `AgentRegistry` `tokenId` (soulbound — cannot be transferred)
2. EIP-712 signer for `CompassHub.consumeGrant` (verified via `recovered == agentRegistry.ownerOf(g.agentTokenId)` — `UnauthorizedSigner` if mismatch)

3. SD-JWT VC `cnf` claim (presentation keybinding)

4. Authwit Grant signer

When v2 lands, a judge can verify “the holder, the agent owner, and the grant signer are the same principal” from public on-chain data + the VC. Today, the on-chain primitive (AgentRegistry + CompassHub) is real; the wallet wiring on the receipt-mint flow is fixture.

Receipt lifecycle (v0.5 — shipped end-to-end)

Both the enclave half (see `enclave/src/`) and the frontend signing flow are real. The full sequence as of v0.5:

1. Provider sends a fresh `challenge` + `policyId` to Maria’s app (currently fixture-supplied; per-NGO providers in v0.7).
2. App signs the EIP-712 Compass `Grant` via Privy’s embedded wallet. The signed grant carries `{agentTokenId, policyId, provider, nonce, expiry, nullifier}`.
3. Browser POSTs `{grant, sig}` to `/api/consume`. The route:
 - Rate-limits the caller (5 req/min/IP, sliding window, in-memory bucket — see `app/src/lib/ratelimit.ts`).
 - Recovers the EIP-712 signer.
 - Computes `agentIdCommitment = keccak256(abi.encode(uint256, address))` matching the contract’s on-chain encoding.
 - Calls the live Phala dstack TDX enclave at `COMPASS_ENCLAVE_URL` with the receipt inputs.
4. The enclave (sealed-image, dstack `getKey('compass-receipt-signer')` derives a deterministic secp256k1 priv inside the attested image):
 - Loads canonical policy JSON for the requested `policyId`.
 - Evaluates the predicate against the receipt inputs.
 - Computes a per-receipt RA quote whose `report_data` commits to `sha256(ethAddress || composeHash || receiptId)` — defeats archived-quote replay across deployments.
 - Constructs the receipt-doc per `docs/schemas/receipt-v1.json`.
 - Signs the canonicalized receipt with the sealed key.
 - Returns `{attestationDigest, signerAddress, perReceiptQuoteHex, receiptVersion}`.
5. `/api/consume` fails closed if the enclave was reachable but did not return a real TDX attestation (`teeSource !== "tee"`). The route never mints a receipt with the placeholder stub digest when an enclave URL is configured — see `app/src/app/api/consume/route.ts`.

6. Provider relayer (`PROVIDER_PRIVATE_KEY`) calls `CompassHub.consumeGrantAndIssueReceipt(grant, sig, receipt)` on the active chain (`activeChain()` returns Aristotle when `NEXT_PUBLIC_COMPASS_USE_MAINNET=1`, Galileo otherwise).
7. The contract validates 8 conditions atomically: `msg.sender == grant.provider`, grant expiry, policy existence + active, nullifier replay, signature length, signer recovery + binding via `agentRegistry.ownerOf(grant.agentTokenId)`, receipt-fields non-zero, receipt expiry, receiptId replay.
8. The same tx emits `GrantConsumed` and `ReceiptIssued`. The `ReceiptIssued` event carries `{receiptId, policyId, nullifier, agentIdCommitment, resultHash, expiry, attestationDigest, timestampBucket}` — no name, HKID, employer, or document fields.
9. The off-chain verifier (`enclave/src/verify-receipt.ts` — `--bundle` mode) re-derives the entire chain locally: signer recovery, quote freshness, image binding, attestation digest. A fake on-chain event with a forged `attestationDigest` fails this check — receipt validity is established off-chain, not by the on-chain log alone. See `docs/honest-limits.md` §18 for the “audit log vs source of truth” design choice.

What’s deployed where

Network	Status
Galileo Testnet (chainId 16602)	Live — AgentRegistry <code>0x461eda452ffAF43c674ef42BdccfDd6B8e13C2D8</code> , CompassHub <code>0x60BbE5fcA6D23f7d25142E721258c641b45A7c3b</code> . All 3 demo policies (HELP, Bethune, Hospital) registered.
Aristotle Mainnet (chainId 16661)	Live (deployed 2026-05-11) — AgentRegistry <code>0xf1FAaBef1d00Db1a15b7637Dc0d8526449D06Bf9</code> , CompassHub <code>0xe42fd4F0a3197126fEeF5e6AAfC5Fb8848bBC58b</code> . All 3 demo policies registered. Frontend uses Aristotle when <code>NEXT_PUBLIC_COMPASS_USE_MAINNET=1</code> .

Composition of Compass primitives outside Track 5

The same architecture serves DV survivors (shelter eligibility), foster youth aging out (housing eligibility), undocumented immigrants (emergency healthcare eligibility), journalists' sources (whistleblower-program eligibility). One protocol, many vulnerable populations. Maria is the lead persona.